APS380 • Driver-in-the-loop battery test platform

Standalone software architecture | October 8, 2026 | Positive cell current = discharge; negative = charge

Solid arrows: working offline files/functions. Dashed arrows: planned integration. Brown arrows: physical effects / protection, not a wiring schematic.

WORKING NOW — Milestone 1: recorded telemetry → estimated EV demand → signed, bounded single-cell request

1 Input recordings

Genuine TORCS CSV: source unchanged
Synthetic CSV: controlled examples
No live TORCS or battery connection

CSV

2 Adapter + validation

curLapTime → seconds; speedX / 3.6
accel → throttle; brake → brake
Split resets / gaps; report exclusions
No invented time or stitched laps

SI data

3 EV demand model

Δv/Δt + interval-average speed
Inertia + rolling + aerodynamic load
Drive losses, auxiliaries, regen cap
Optional offline smoothing for studies

EV watts

4 Cell scaling + limits

Nominal cell / pack energy ratio
Optional experiment reduction factor
Separate charge / discharge limits
Limits are offline examples

Cell amps

5 Evidence and outputs

CSV: requested versus limited current
Plots and effective configuration
19 tests; smoothing / energy studies
No physical current is commanded

Configuration is explicit. Each continuous segment is processed independently. The first sample assumes zero acceleration and has zero energy-integration duration.

PLANNED CLOSED LOOP — Python decides demand and capability; ESP32 executes locally; PCB protection remains independent

TORCS + Logitech G29

G29 USB input → steering / pedals
TORCS C/C++ simulates vehicle motion
Exports timestamped driving telemetry
Applies vehicle power / torque limits
Exporter and feedback hook unverified

Driving telemetry
CSV first; UDP later

Allowed vehicle power
Proposed localhost UDP

Python: demand + feedback

Validate telemetry → EV power → cell scale
Use one allowable-power decision for:
• signed cell-current request
• equivalent vehicle power / torque limit
Do not scale steering or all pedals together

USB serial: signed amps,
ARM / DISARM, heartbeat

Measured V + separate SOP
feed the common limiter

Demand / decisions
feed logs

ESP32 DevKit firmware

Validate host requests against local limits
ARM / DISARM; heartbeat timeout; faults
Acquire sensors; map current to eLoad
Drive disconnect through driver circuitry
Report actual measurements and state

USB serial: timestamped
V / I / T, state and faults

Sensor samples
I²C / ADC

PCB measurements

Cell voltage and temperature
Bidirectional NET cell current
Proposed I²C monitor / ADC
INA228 / ADS1115 not final
Pins / addresses not assigned

Physical cell
conditions

Python: battery state + serial

SOC (state of charge): net current counting
Measured V / I / T + SOC → separate SOP
SOP = available charge / discharge power
USB serial sends requests; receives status
Logs commands separately from measurements

Fault indication if available
Protection does not wait for MCU

BK: verified serial interface
PSU remote control optional

Independent PCB protection

Fuse; over / undervoltage protection
Overcurrent / temperature trips
Emergency stop; default-open isolation
Works without Python or ESP32
May expose a fault-status signal

GPIO via driver
Disconnect request

Independent
physical trip

Instruments + cell

BK 8500: CC load draws current
PSU supplies current; manual first
Combined branches stress 18650
Net current must be measured
Controlled battery disconnect

ON-PAGE DESIGN NOTES — definitions, assumptions and unresolved interfaces

Vehicle and current equations

a = Δv / Δt; v = mean interval speed
Pwheel = (m a + m g Crr + ½ ρ CdA v²) v
Drive: Pbattery = Pwheel / ηdrive + Paux
Braking: −capped recovery × ηregen + Paux
k = (Vnom × capacityAh / packWh) × scale
Pcell = k × Pbattery; Ioffline = Pcell / Vnom
Clamp to [−Icharge_limit, +Idischarge_limit]

Live battery decisions and feedback

SOCnext = SOC − Imeasured Δt / (3600 Ah)
Use measured NET current, not commands
SOP supplies separate charge / discharge bounds
Allowed power also respects cell current limits
Irequest = Pallowed / valid measured voltage
Use the SAME effective k for vehicle feedback
Handle zero demand and sign transitions explicitly

PSU / eLoad mapping is conditional

Icell = IeLoad − IPSU at the common network
Only if constant PSU bias is verified:
IeLoad_request = Ibias + Icell_request
PSU current limit is not measured PSU current
Load requests must be nonnegative / achievable
Zero software amps do not guarantee zero current
Protection must prevent unintended PSU charging

Implementation boundaries

Python has no hardware commands today
USB serial format / timing remain to be designed
BK interface / isolation / low-voltage limits unverified
PSU model unknown; remote control is optional
A GUI and a separate BMS module are not required
Local watchdog does not replace PCB protection
PC → eLoad is only a fallback if ESP32 link fails

What the current profiles establish

Genuine-source ingestion and repeatable equations / limits are verified. Power is estimated, not measured.
The likely source-car model uses 1150 kg and body CdA = 0.672 m²; wings, grade and slip are omitted.
Smoothing and experiment scaling are explicit study choices; a 0.2 factor sacrifices nominal C-rate equivalence.
Example current limits are not approved cell ratings. This architecture is not a wiring or protection schematic.

Next milestones

Select and justify a demonstration profile → verify sensor/action timing → record this team’s TORCS / G29 data.
Choose the exact cell and approved envelope → mock USB serial → validate ESP32 sensing and instrument control.
Then add measured-current SOC, separate SOP limits and vehicle derating from the common power calculation.
Independent PCB protection requires its own implementation and validation throughout hardware development.